

torch.addmm#

<https://pytorch.org/docs/stable/generated/torch.addmm.html>

Description:

Performs a matrix multiplication of the matrices `mat1` and `mat2`. The matrix input is added to the final result. If `mat1` is a $(n \times m)$ tensor, `mat2` is a $(m \times p)$ tensor, then input must be broadcastable with a $(n \times p)$ tensor and out will be a $(n \times p)$ tensor. `alpha` and `beta` are scaling factors on matrix-vector product between `mat1` and `mat2` and the added matrix input respectively. If `beta` is 0, then the content of input will be ignored, and `nan` and `inf` in it will not be propagated. For inputs of type `FloatTensor` or `DoubleTensor`, arguments `beta` and `alpha` must be real numbers, otherwise they should be integers.

Function Signature

```
torch.addmm(input, mat1, mat2, out_dtype=None, *, beta=1,
alpha=1, out=None) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

β (Number, optional) – multiplier for input (β) α (Number, optional) – multiplier for $\text{mat1} @ \text{mat2}$ (α) out (Tensor, optional) – the output tensor.

Code Examples

```
>>> M = torch.randn(2, 3)
>>> mat1 = torch.randn(2, 3)
>>> mat2 = torch.randn(3, 3)
>>> torch.addmm(M, mat1, mat2)
tensor([[ -4.8716,  1.4671, -1.3746],
        [ 0.7573, -3.9555, -2.8681]])
```

Notes & Warnings

WARNING

Warning Sparse support is a beta feature and some layout(s)/dtype/device combinations may not be supported, or may not have autograd support. If you notice missing functionality please open a feature request.

Detailed Documentation

Documentation

Performs a matrix multiplication of the matrices `mat1` and `mat2`. The matrix input is added to the final result.

If `mat1` is a $(n \times m)(n \times m)$ tensor, `mat2` is a $(m \times p)(m \times p)$ tensor, then input must be broadcastable with a $(n \times p)(n \times p)$ tensor and out will be a $(n \times p)(n \times p)$ tensor.

`alpha` and `beta` are scaling factors on matrix-vector product between `mat1` and `mat2` and the added matrix input respectively.

If `beta` is 0, then the content of input will be ignored, and nan and inf in it will not be propagated.

For inputs of type `FloatTensor` or `DoubleTensor`, arguments `beta` and `alpha` must be real numbers, otherwise they should be integers.

This operation has support for arguments with sparse layouts. If input is sparse the

result will have the same layout and if out is provided it must have the same layout as input.

Sparse support is a beta feature and some layout(s)/dtype/device combinations may not be supported, or may not have autograd support. If you notice missing functionality please open a feature request.

This operator supports TensorFloat32.

On certain ROCm devices, when using float16 inputs this module will use different precision for backward.

input (Tensor) – matrix to be added

mat1 (Tensor) – the first matrix to be matrix multiplied

mat2 (Tensor) – the second matrix to be matrix multiplied

out_dtype (dtype, optional) – the dtype of the output tensor, Supported only on CUDA and for torch.float32 given torch.float16/torch.bfloat16 input dtypes

beta (Number, optional) – multiplier for input (β)

alpha (Number, optional) – multiplier for $\text{mat1} @ \text{mat2}$ (α)

out (Tensor, optional) – the output tensor.